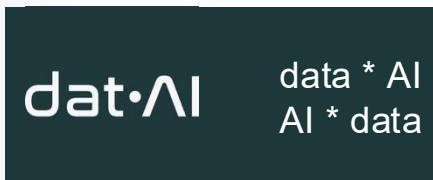


Matteo Palmonari

matteo.palmonari@unimib.it

Word Embeddings



Word Embeddings & Word2Vec: Outline

- Main idea behind Word2vec
- Word2vec: Skip-gram and CBOW
- Word2vec: CBOW in details
- Word Embeddings: Similarity & Regularities

Word Embeddings & Word2Vec: Main Idea

A first look to Word2Vec

- Vector representations of words, i.e., word embeddings, are generated from a text corpus using a neural network [Mikolov+, 2013]



My

little	black	cat	sleeps	all
--------	-------	-----	--------	-----

 day.

→ The

big	black	cat	eats	its
-----	-------	-----	------	-----

 food. →

Sometimes	my	dog	eats	too
-----------	----	-----	------	-----

 much!



$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- Similar words** appear in similar contexts and **have similar vectors**
- The neural network generates vectors so as to predict a target word given its context, or, a context given a target word

Word2vec – Predicting Words Nearby with DNNs

Intuition. Given a specific word in the middle of a sentence (the input word or center word), look at the words **nearby** and pick one at random. The **network** is going to tell us the probability for every word in our vocabulary of being the “nearby word” that we chose.”



My

little	black	cat	sleeps	all
--------	-------	-----	--------	-----

 day.

→ The

big	black	cat	eats	its
-----	-------	-----	------	-----

 food. →

Sometimes	my	dog	eats	too
-----------	----	-----	------	-----

 much!

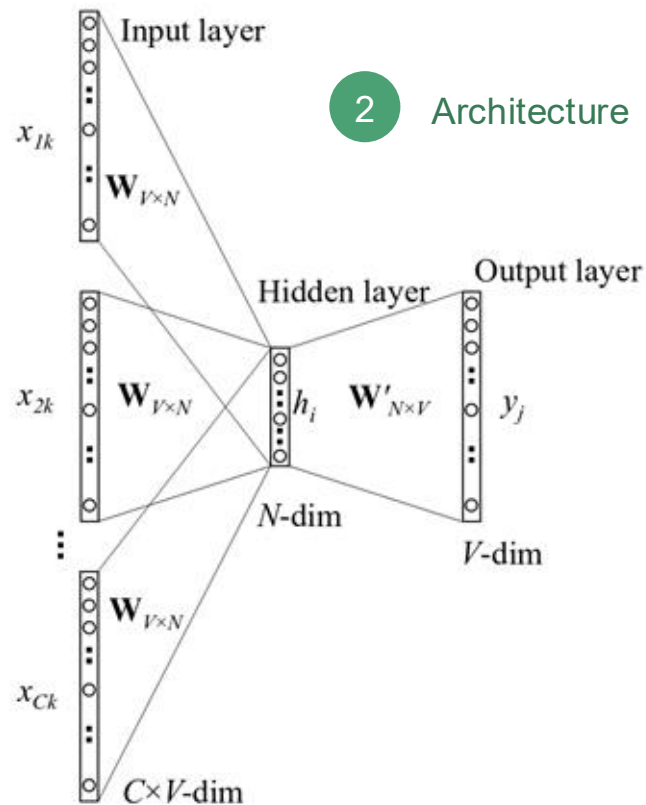


$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- Similar words appear in similar contexts and have similar vectors
- The neural network generates vectors so as to predict a target word given its context, or, a context given a target word

Word Embeddings & Word2Vec: CBOW & Skip-Gram

word2vec – Continuous Bag-of-word (CBOW)



Objective Function for CBOW:

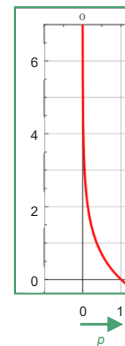
1

$$\text{maximize } P(w_c | w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m})$$

$$\text{minimize } -\log P(w_c | w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m})$$

where

w_c is the center word and
 m is the window size



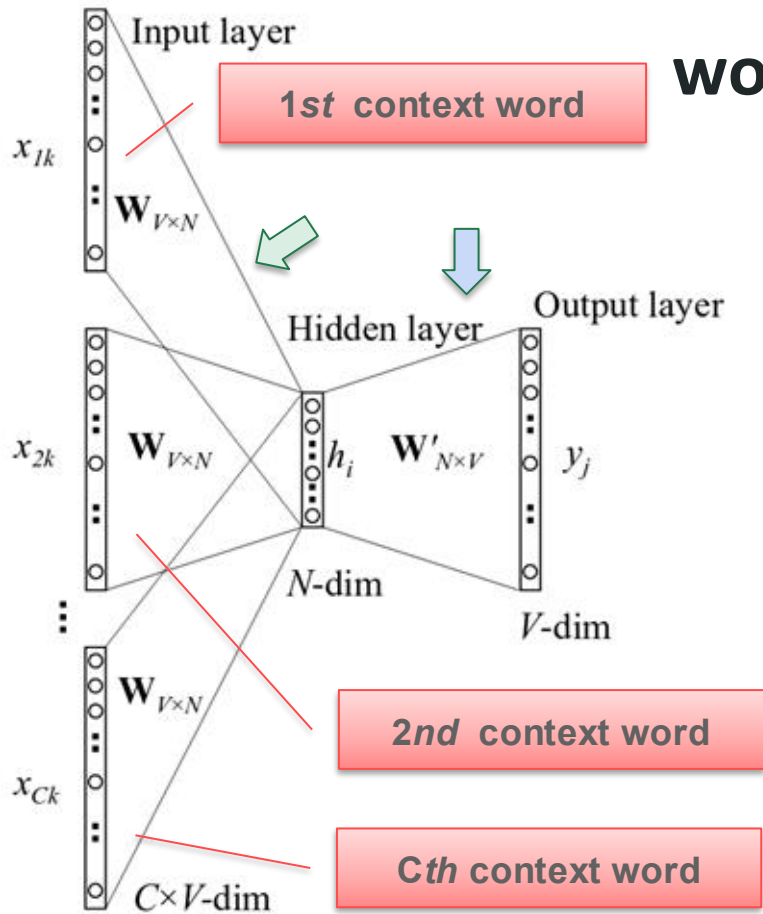
$-\ln(x)$ for $0 < x < 1$
 ...e.g., $x = p(w_c)$

Two matrices:

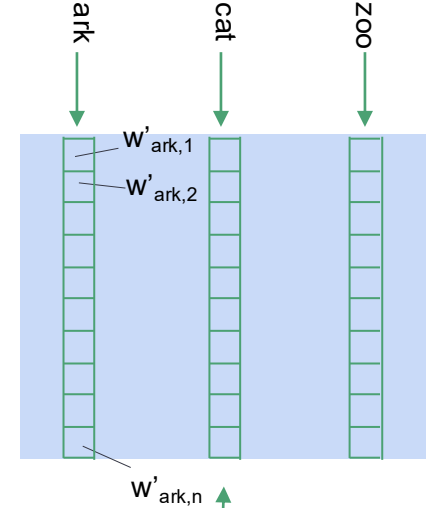
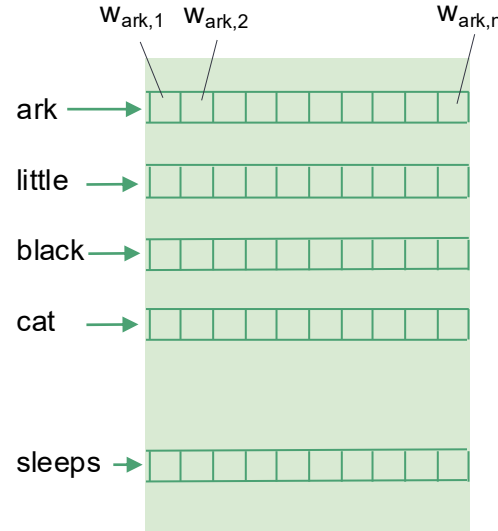
- Input to hidden layer (large to small)
- Hidden layer to output (small to large)

Figure 2: Continuous bag-of-word model

word2vec – CBOW



$$\begin{aligned} \mathbf{h} &= \frac{1}{C} \mathbf{W}^T (\mathbf{x}_1 + \mathbf{x}_2 + \dots + \mathbf{x}_C) \\ &= \frac{1}{C} (\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \dots + \mathbf{v}_{w_C})^T \end{aligned}$$

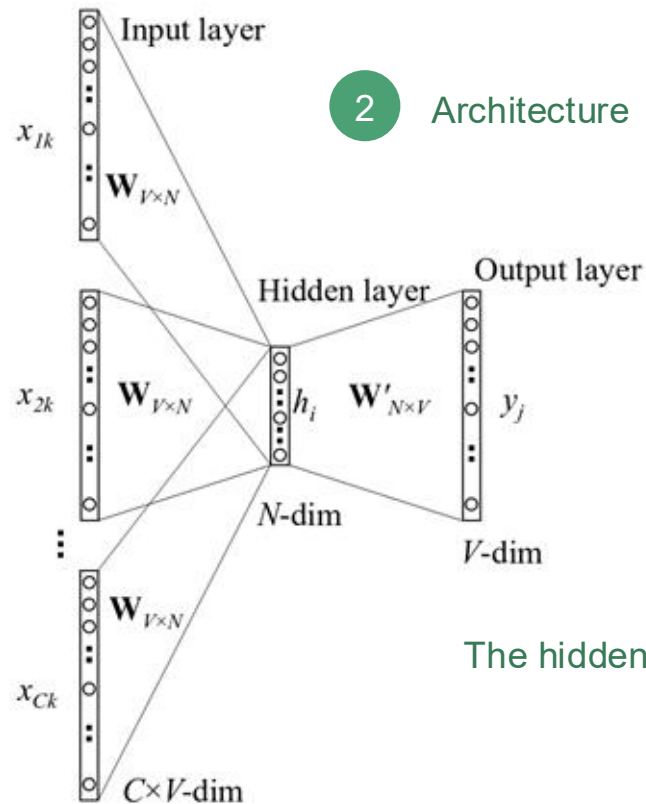


$\mathbf{W}'$ ($N \times V$)
This matrix contains **word embeddings**

$\mathbf{W}$ ($V \times N$)
This matrix contains **context embeddings**
(embeddings of the word when they are used as a context)

Figure 2: Continuous bag-of-words model

word2vec – Continuous Bag-of-word (CBOW)



The hidden layer returns the average context embedding

$$\begin{aligned} \mathbf{h} &= \frac{1}{C} \mathbf{W}^T (\mathbf{x}_1 + \mathbf{x}_2 + \dots + \mathbf{x}_C) \\ &= \frac{1}{C} (\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \dots + \mathbf{v}_{w_C})^T \end{aligned}$$

Objective Function for CBOW:

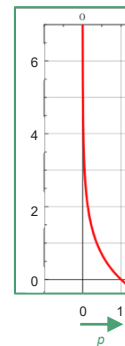
1

$$\text{maximize } P(w_c | w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m})$$

$$\text{minimize } -\log P(w_c | w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m})$$

where

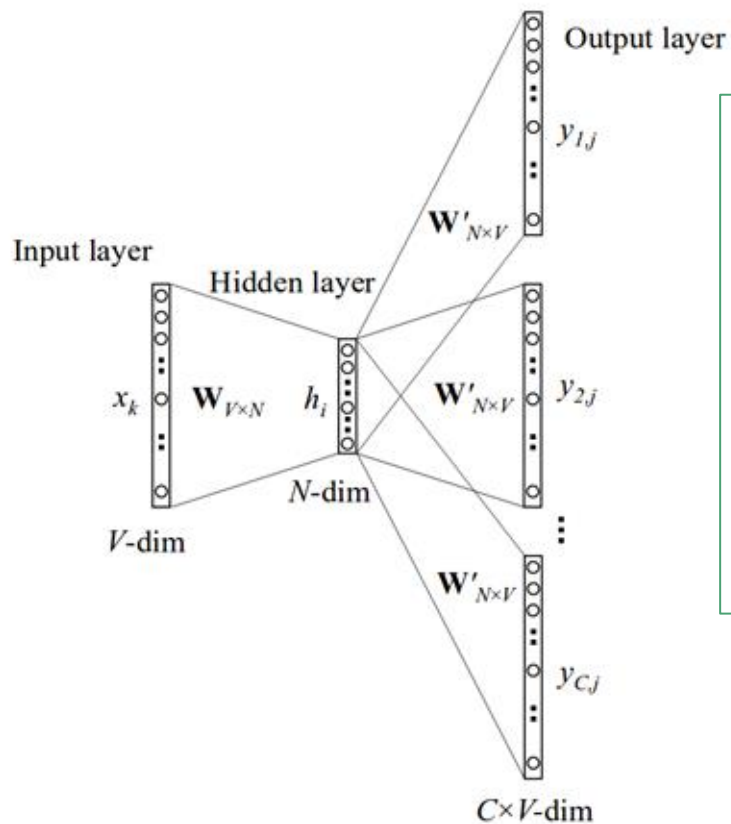
w_c is the center word and
 m is the window size



$-\ln(x)$ for $0 < x < 1$
 ...e.g., $x = p(w_c)$

Figure 2: Continuous bag-of-word model

word2vec: Skip-Gram



Objective Function for Skip-Gram:

$$\text{maximize } P(w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m} \mid w_c)$$

$$\text{minimize } -\log P(w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m} \mid w_c)$$

where

w_c is the center word and
 m is the window size

<https://arxiv.org/pdf/1411.2738.pdf>

Figure 3: The skip-gram model.

word2vec – Skip-Gram

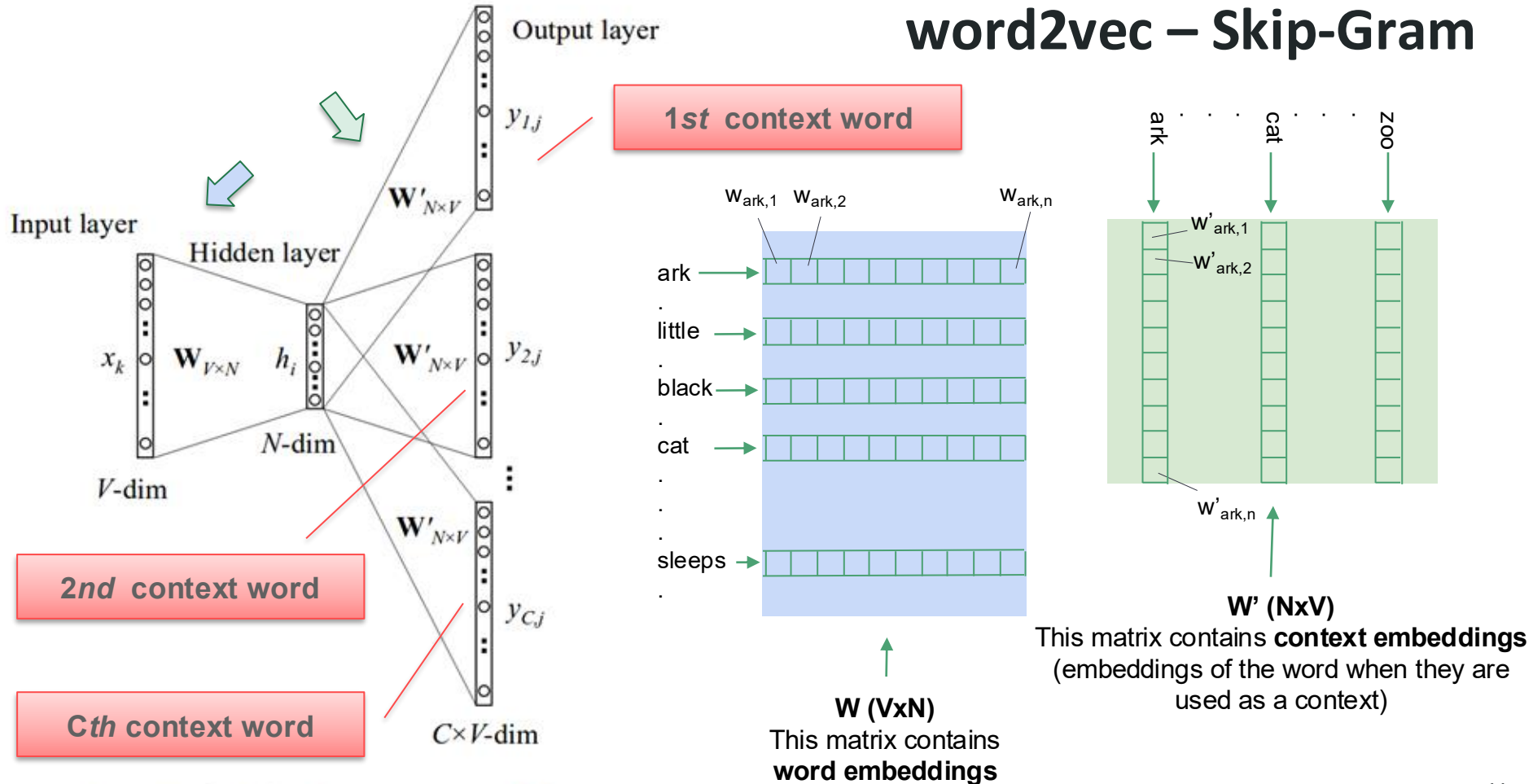


Figure 3: The skip-gram model.

word2vec - Skip-Gram: Training

Source Text

Training
Samples

The quick brown fox jumps over the lazy dog. →

(the, quick)
(the, brown)

The quick brown fox jumps over the lazy dog. →

(quick, the)
(quick, brown)
(quick, fox)

The quick brown fox jumps over the lazy dog. →

(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)

The quick brown fox jumps over the lazy dog. →

(fox, quick)
(fox, brown)
(fox, jumps)
(fox, over)

Nearby means that we should define a window that spans over the text

**(input, output)
training pair**

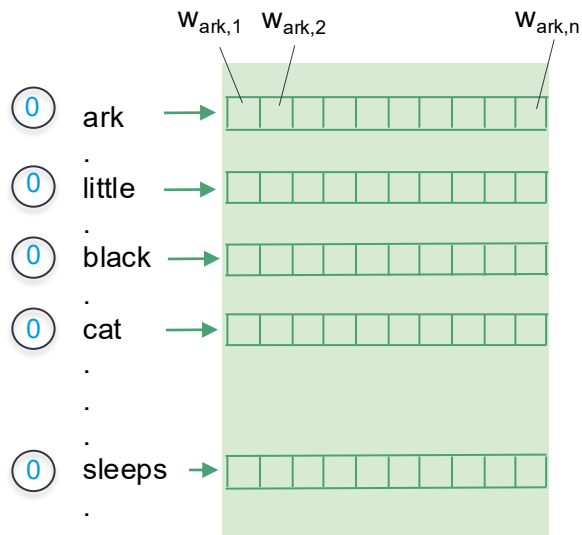
word2vec is going to learn statistics about the word relations

Word Embeddings & Word2Vec: CBOW in detail

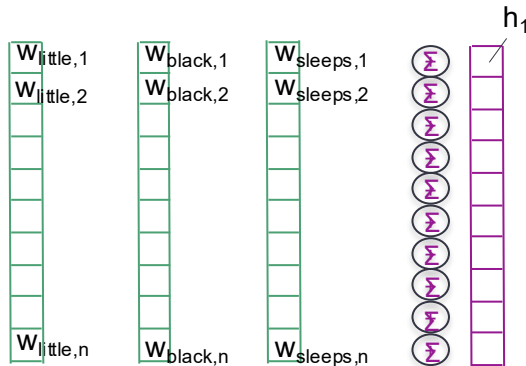
CBOW Details: Inizialization

W (VxN)

This hidden layer weight matrix contains context embeddings



W^T
the transposed
weight matrix for non-0 input

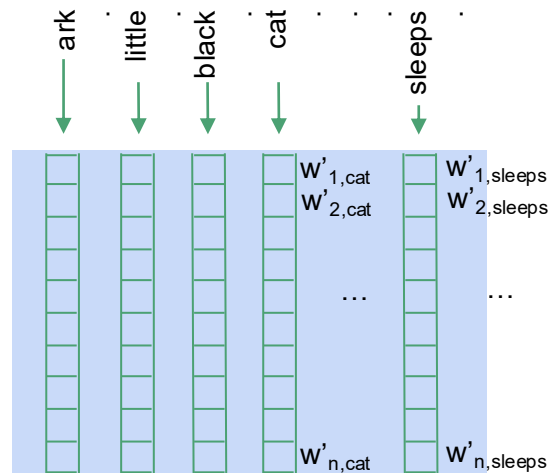


Hidden layer and output (**h**)

$$\begin{aligned} \mathbf{h} &= \frac{1}{C} \mathbf{W}^T (\mathbf{x}_1 + \mathbf{x}_2 + \dots + \mathbf{x}_C) \\ &= \frac{1}{C} (\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \dots + \mathbf{v}_{w_C})^T \end{aligned}$$

W' (NxV)

This output layer weight matrix contains word embeddings



Output layer



p_{ark} p_{little} p_{black} p_{cat} p_{sleeps}

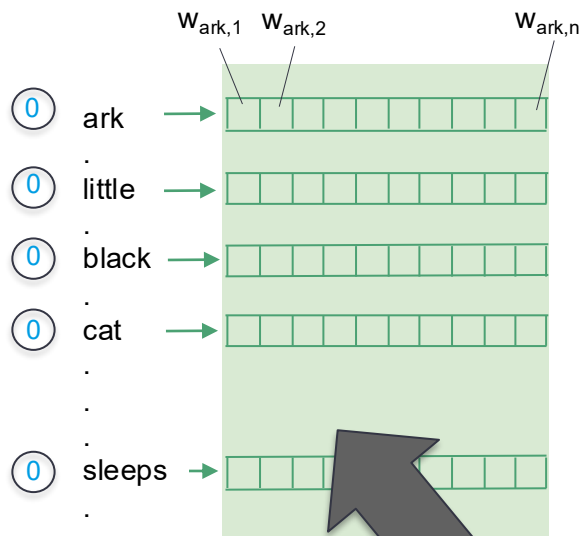
Probability
distribution

Input
layer
(one hot
encoding)

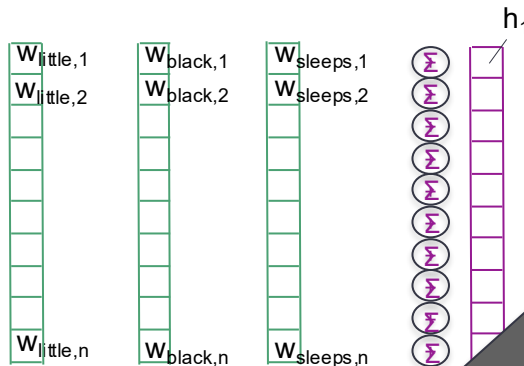
CBOW Details: Inizialization

W ($V \times N$)

This hidden layer weight matrix contains context embeddings

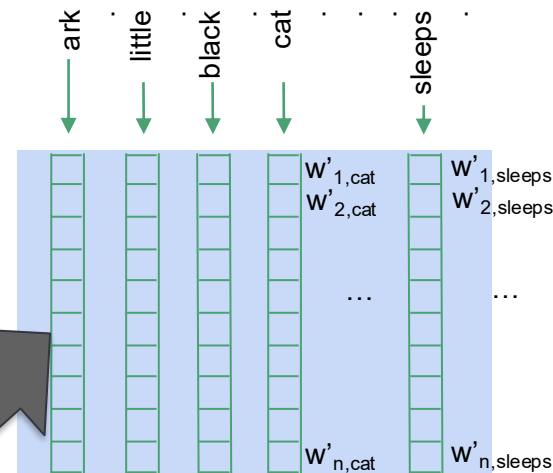


W^T
the transposed
weight matrix for non-0 input



W' ($N \times V$)

This output layer weight matrix contains word embeddings



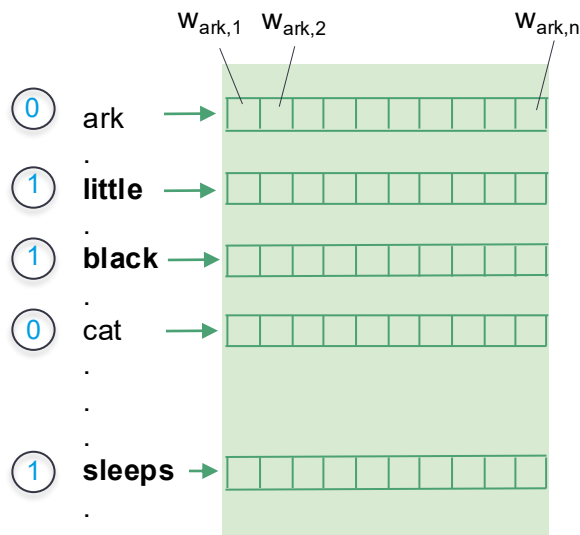
Probability
distribution

*Weights in the matrices W' and W
are initialized*

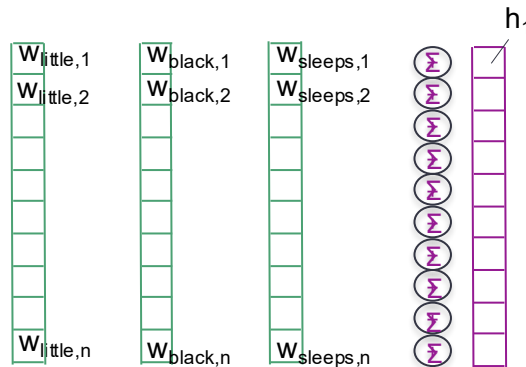
CBOW Details: Forward Pass

$W (V \times N)$

This hidden layer weight matrix contains context embeddings



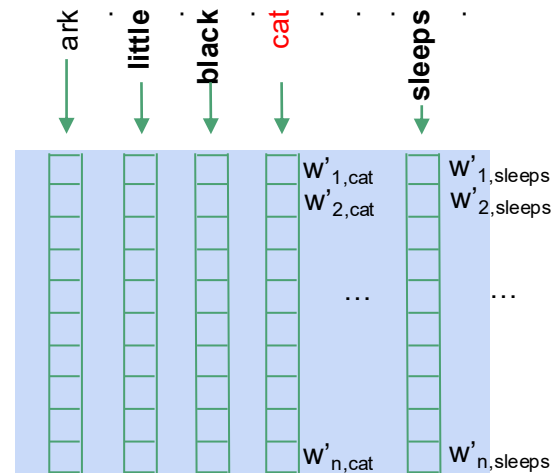
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (h)

$W' (N \times V)$

This output layer weight matrix contains word embeddings



Output layer



p_{ark} p_{little} p_{black} p_{cat} p_{sleeps}

Probability
distribution

... little black **cat** sleeps.

Context words

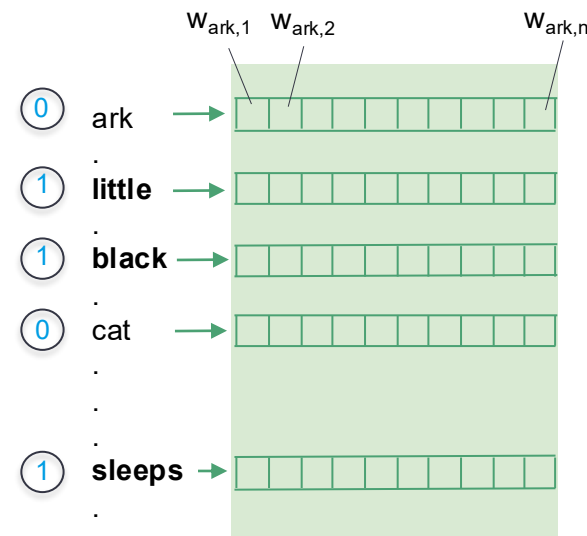
Target word

Input
layer
(one hot
encoding)

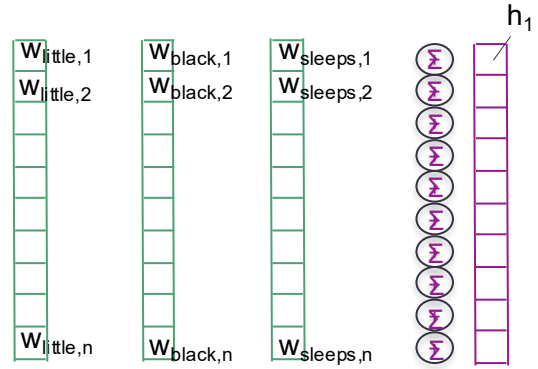
CBOW Details: Forward Pass

W (VxN)

This hidden layer weight matrix contains context embeddings



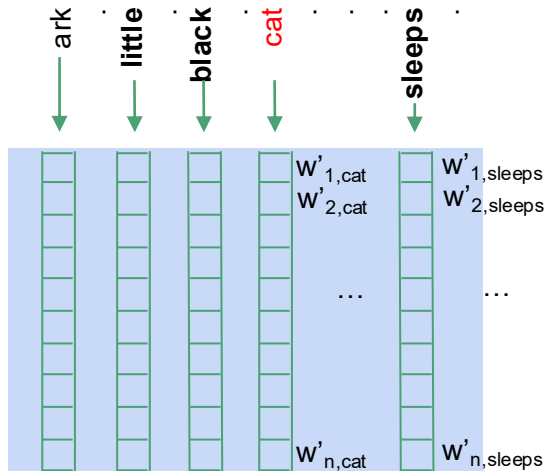
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (**h**)

W' (NxV)

This output layer weight matrix contains word embeddings



Output layer



p_{ark} p_{little} p_{black} p_{cat} p_{sleeps}

Probability
distribution

... little black **cat** sleeps.

Context words

Target word

Input
layer
(one hot
encoding)

Side Note: Lookup Matrix

One hot encoded vectors allows us to do lookup operations

This means that when we multiply a one-hot encoded vector of a word with the matrix we access to its embedding

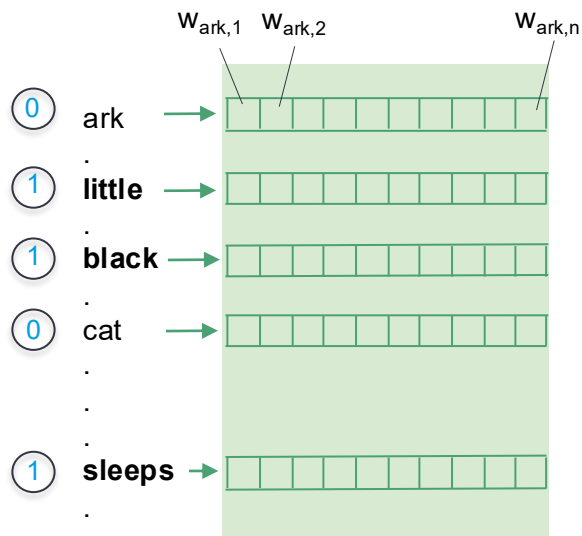
$$(1 \ 0 \ 0 \ 0 \ 0) \cdot \begin{pmatrix} 1 & 2 & 3 & 7 \\ 1 & 7 & 2 & 8 \\ 3 & 1 & 2 & 3 \\ 9 & 3 & 4 & 2 \\ 6 & 1 & 2 & 1 \end{pmatrix} = \{1, 2, 3, 7\}$$

$$(0 \ 1 \ 0 \ 0 \ 0) \cdot \begin{pmatrix} 1 & 2 & 3 & 7 \\ 1 & 7 & 2 & 8 \\ 3 & 1 & 2 & 3 \\ 9 & 3 & 4 & 2 \\ 6 & 1 & 2 & 1 \end{pmatrix} = \{1, 7, 2, 8\}$$

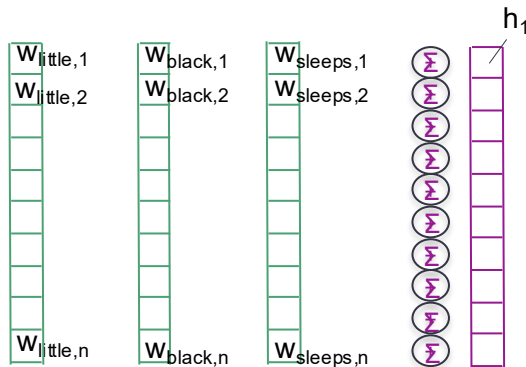
CBOW Details: Forward Pass

$W (V \times N)$

This hidden layer weight matrix contains context embeddings



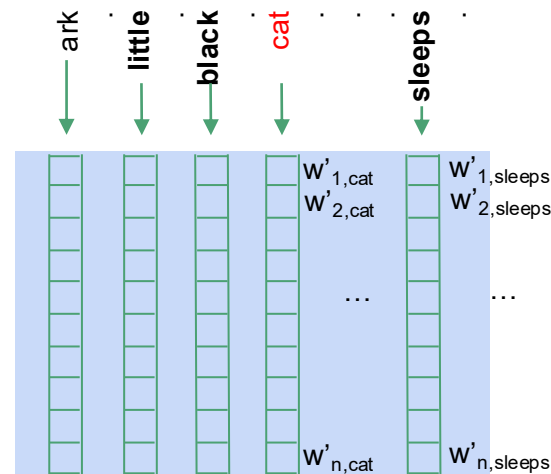
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (h)

$W' (N \times V)$

This output layer weight matrix contains word embeddings



Output layer



p_{ark} p_{little} p_{black} p_{cat} p_{sleeps}

Probability
distribution

... little black **cat** sleeps.

Context words

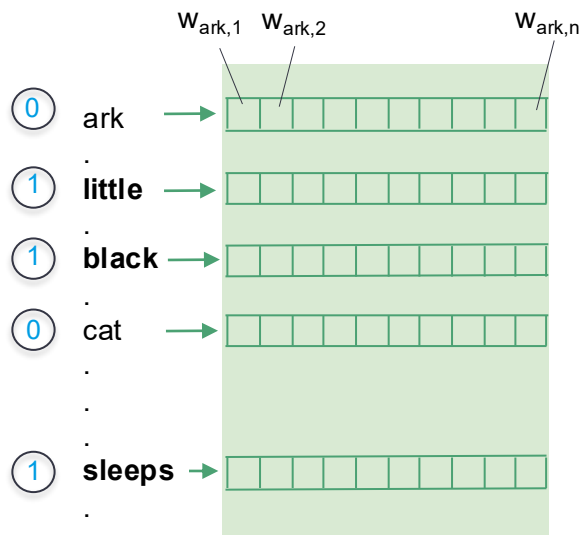
Target word

Input
layer
(one hot
encoding)

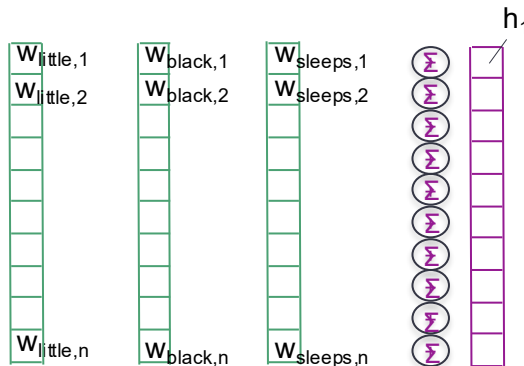
CBOW Details: Forward Pass

$W (V \times N)$

This hidden layer weight matrix contains context embeddings



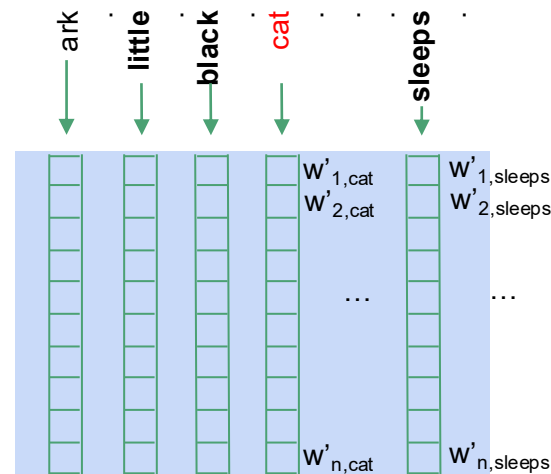
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (h)

$W' (N \times V)$

This output layer weight matrix contains word embeddings



Output layer



p_{ark} p_{little} p_{black} p_{cat} p_{sleeps}

Probability
distribution

... little black **cat** sleeps.

Context words

Target word

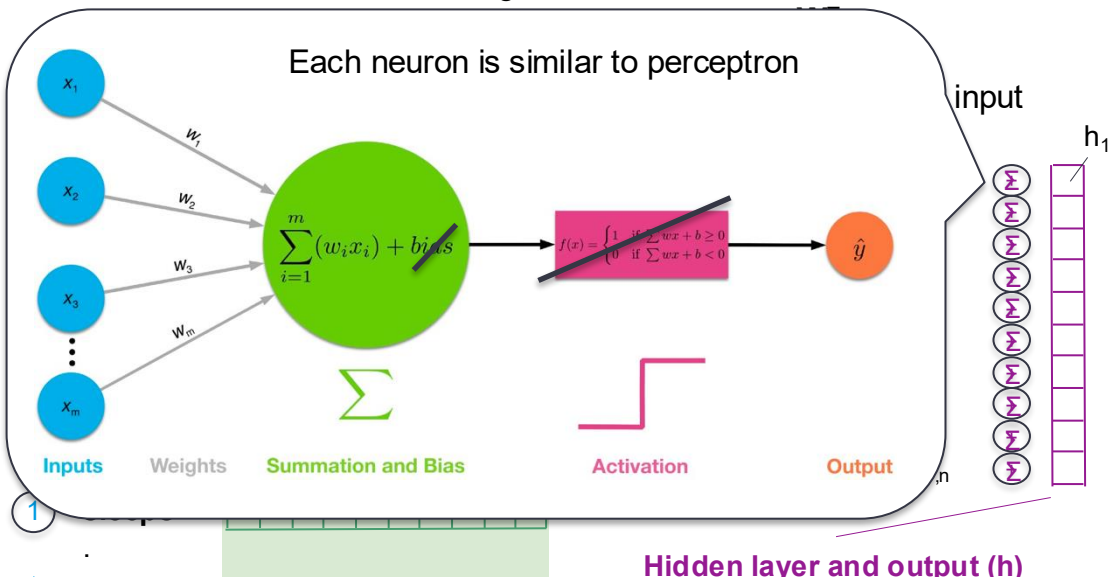
Input
layer
(one hot
encoding)

CBOW Details: Forward Pass

W (VxN)

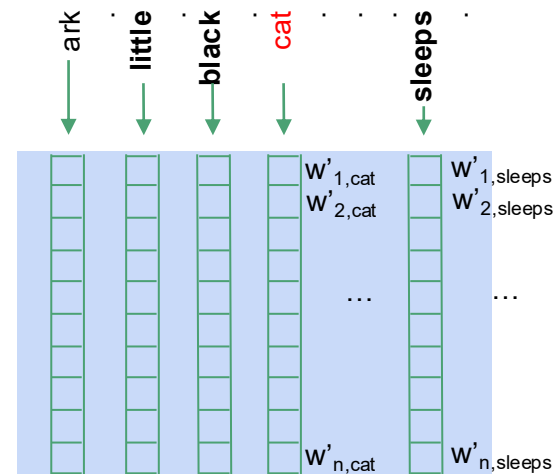
This hidden layer weight matrix contains context embeddings

Each neuron is similar to perceptron



W' (NxV)

This output layer weight matrix contains word embeddings



Output layer



Probability distribution



... little black **cat** sleeps.

Context words

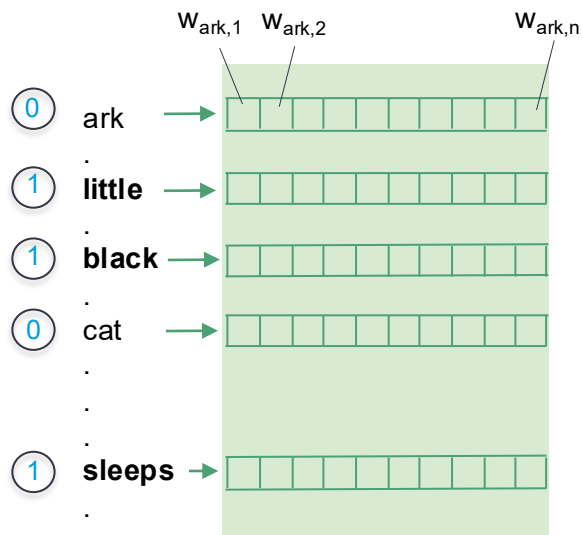
Target word

Input layer
(one hot encoding)

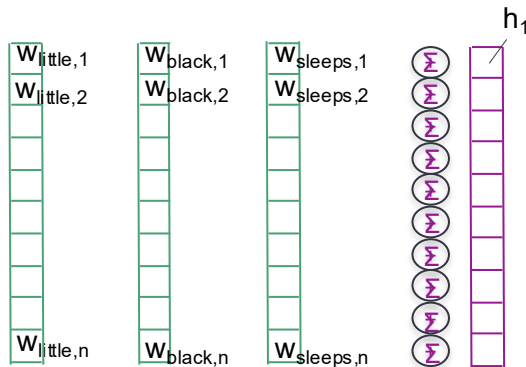
CBOW Details: Forward Pass

$W (V \times N)$

This hidden layer weight matrix contains context embeddings



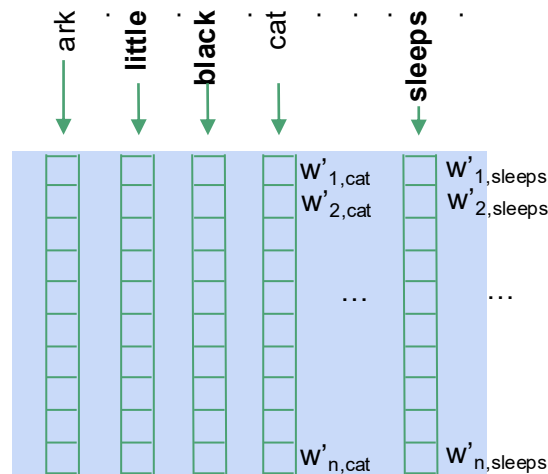
W^T
the transposed
weight matrix for non-0 input



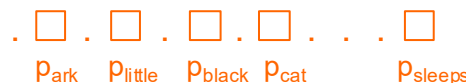
Hidden layer and output (h)

$W' (N \times V)$

This output layer weight matrix contains word embeddings



Output layer



Probability
distribution

... little black **cat** sleeps.

Context words

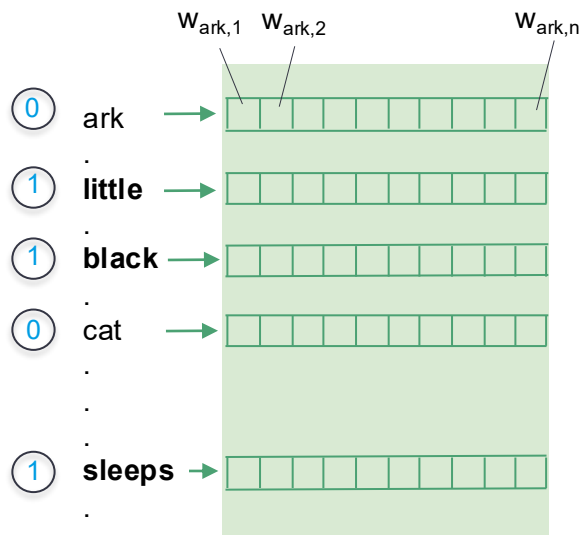
Target word

Input
layer
(one hot
encoding)

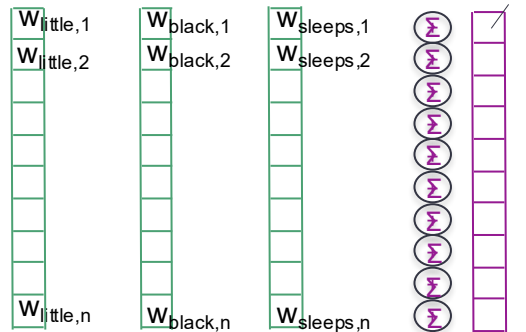
CBOW Details: Forward Pass

W (VxN)

This hidden layer weight matrix contains context embeddings



W^T
the transposed
weight matrix for non-0 input

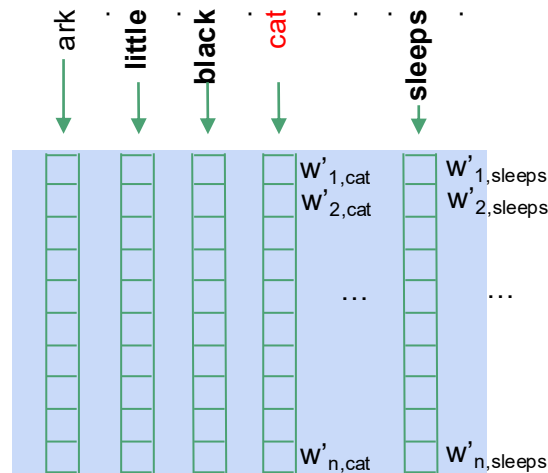


$$h_1 = (1 * w_{\text{little},1} + 1 * w_{\text{black},1} + 1 * w_{\text{sleeps},1}) / 3$$

$$\dots (+ 0 * w_{\text{ark},1} + 0 * w_{\text{cat}} + \dots + 0 * w_v + \dots) / 3$$

W' (NxV)

This output layer weight matrix contains word embeddings



Hidden layer and output (h)

Output layer



$p_{\text{ark}}, p_{\text{little}}, p_{\text{black}}, p_{\text{cat}}, p_{\text{sleeps}}$

**Probability
distribution**

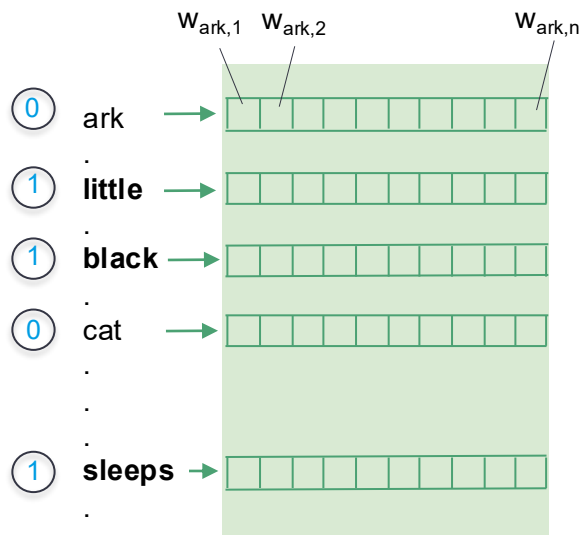
Input
layer
(one hot
encoding)

$$\begin{aligned} \mathbf{h} &= \frac{1}{C} \mathbf{W}^T (\mathbf{x}_1 + \mathbf{x}_2 + \dots + \mathbf{x}_C) \\ &= \frac{1}{C} (\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \dots + \mathbf{v}_{w_C})^T \end{aligned}$$

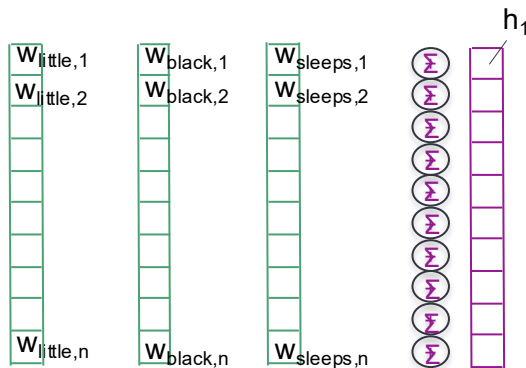
CBOW Details: Forward Pass

$W (V \times N)$

This hidden layer weight matrix contains context embeddings



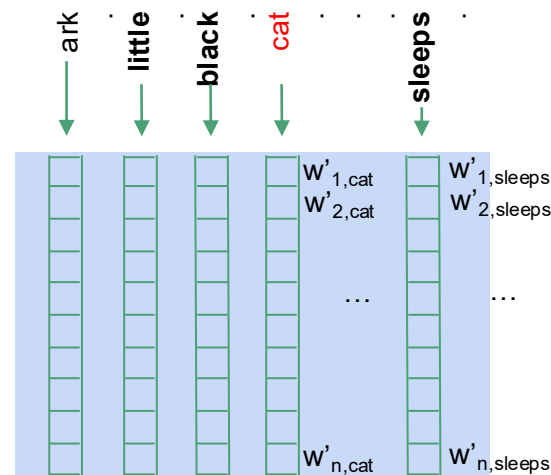
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (h)

$W' (N \times V)$

This output layer weight matrix contains word embeddings



Output layer



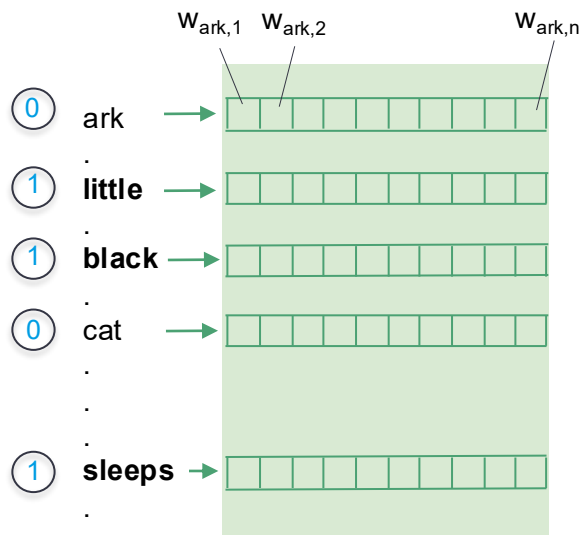
Probability
distribution

Input
layer
(one hot
encoding)

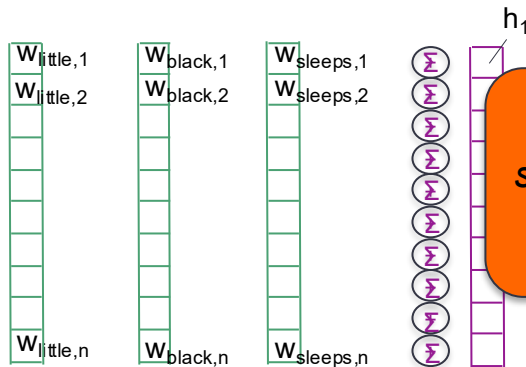
CBOW Details: Forward Pass

W (VxN)

This hidden layer weight matrix contains context embeddings

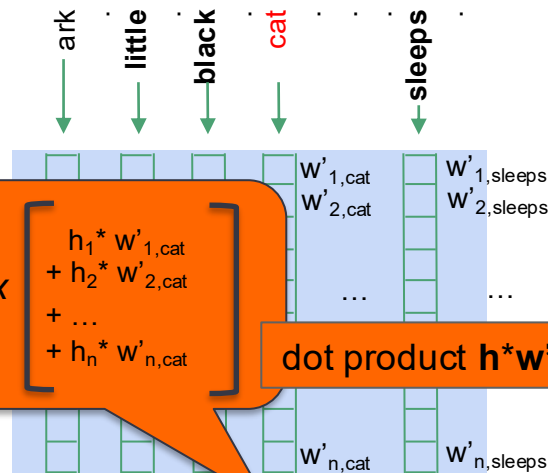


W^T
the transposed
weight matrix for non-0 input



W' (NxV)

This output layer weight matrix contains word embeddings



softmax

$$\begin{aligned} &h_1 * w'_{1,cat} \\ &+ h_2 * w'_{2,cat} \\ &+ \dots \\ &+ h_n * w'_{n,cat} \end{aligned}$$

dot product $h * w'_{cat}$

Hidden layer and output (h)

Output layer

softmax

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \text{for } j = 1, \dots, K$$

($K = V$)



p_{ark} p_{little} p_{black} p_{cat} p_{sleeps}

Probability
distribution

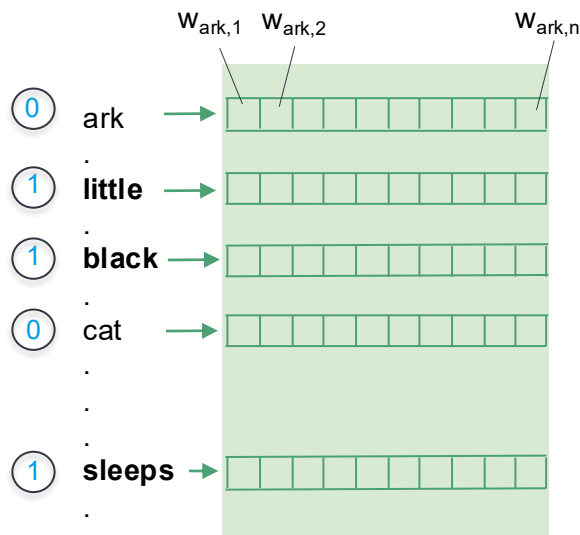
Nice explanation of softmax at:

<https://victorzhou.com/blog/softmax/>

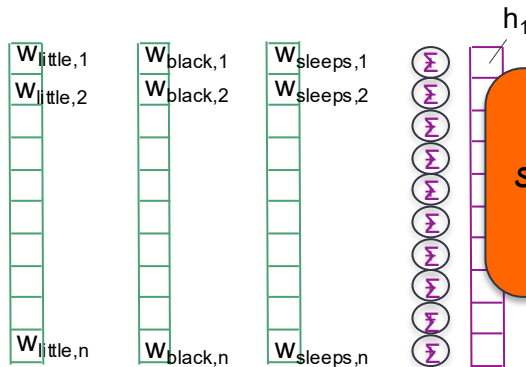
CBOW Details: Forward Pass

$W (V \times N)$

This hidden layer weight matrix contains context embeddings



W^T
the transposed
weight matrix for non-0 input



softmax

$$\begin{aligned} &h_1 * w'_{1,cat} \\ &+ h_2 * w'_{2,cat} \\ &+ \dots \\ &+ h_n * w'_{n,cat} \end{aligned}$$

dot product $h * w'_{cat}$

Hidden layer and output (h)

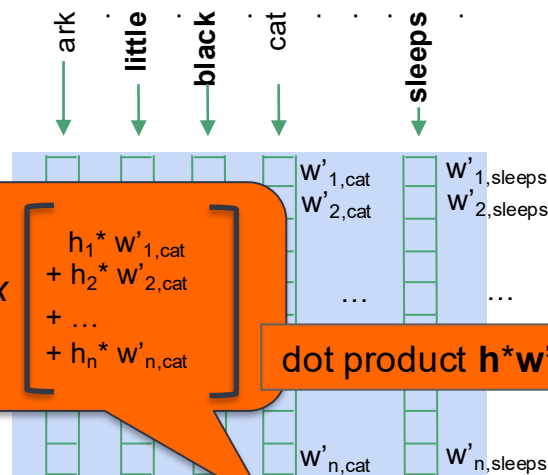
Output layer

softmax

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \text{for } j = 1, \dots, K \quad (K = V)$$

$W' (N \times V)$

This output layer weight matrix contains word embeddings



$K = V$ may be very large, e.g. $>200k$ for an English vocabulary!

More efficient training strategies

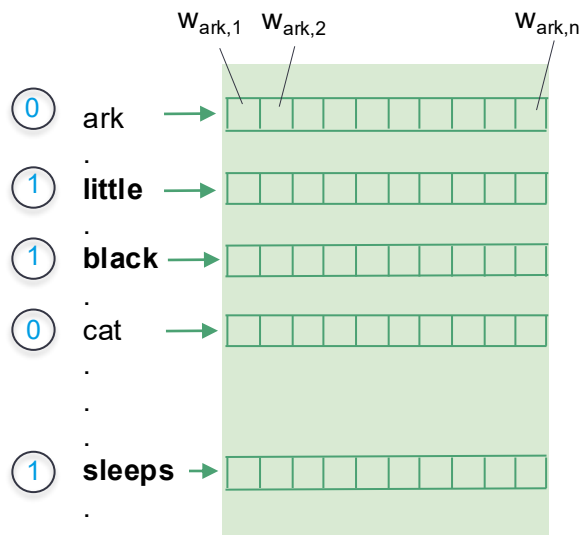
proposed, (e.g., negative sampling: from probability-based task to similarity-based task) – see additional resources

Input layer
(one hot encoding)

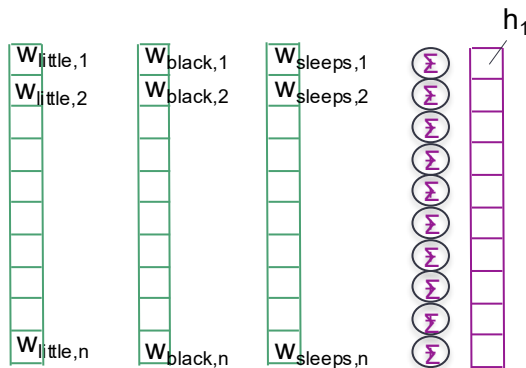
CBOW Details: Training Example

W (VxN)

This hidden layer weight matrix contains context embeddings



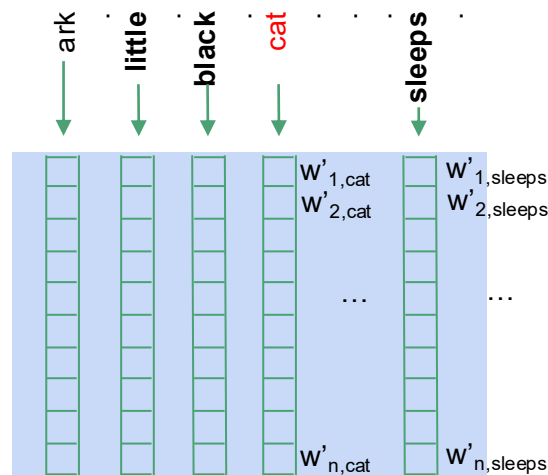
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (h)

W' (NxV)

This output layer weight matrix contains word embeddings



Output layer

softmax

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \text{for } j = 1, \dots, K$$

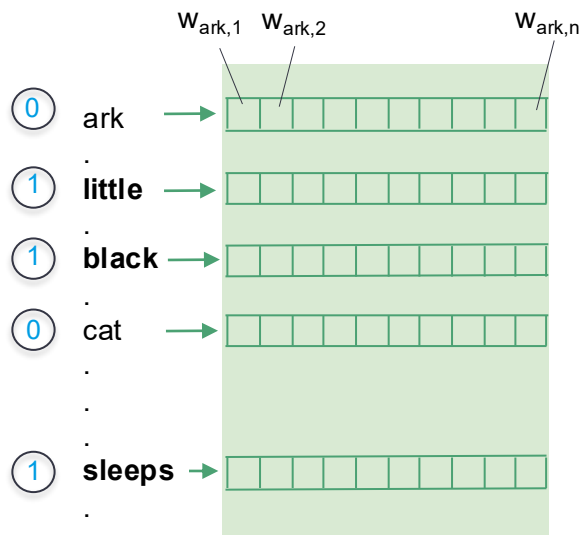


Input layer
(one hot encoding)

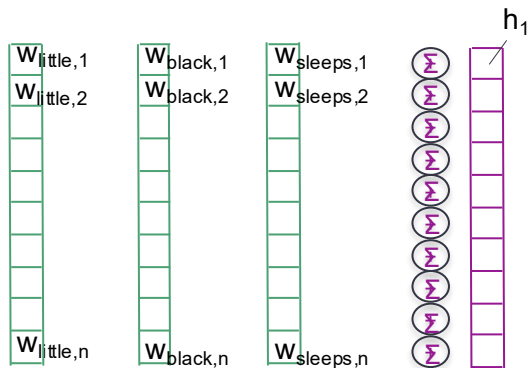
CBOW Details: Loss Function

W (VxN)

This hidden layer weight matrix contains context embeddings



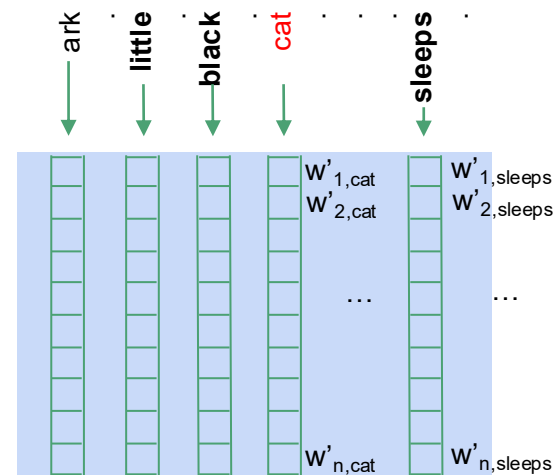
W^T
the transposed
weight matrix for non-0 input



Hidden layer and output (h)

W' (NxV)

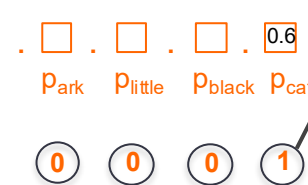
This output layer weight matrix contains word embeddings



Output layer

softmax

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \text{for } j = 1, \dots, K$$



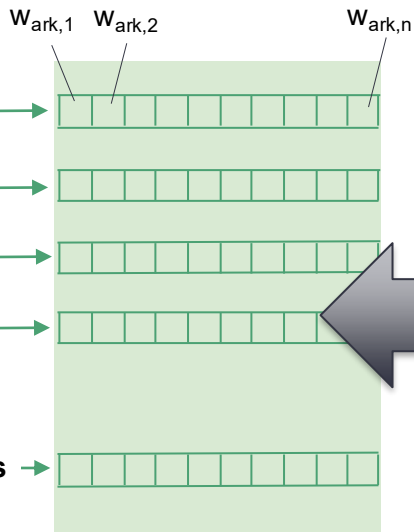
**loss
(distance)**

Input layer
(one hot encoding)

CBOW Details: Backpropagation

W (VxN)

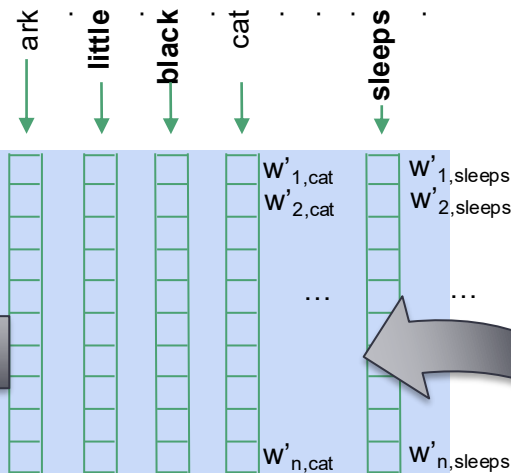
This hidden layer weight matrix contains context embeddings



Weights in the matrices W' and W are updated via backpropagation to decrease the loss

W' (NxV)

This output layer weight matrix contains word embeddings



Hidden layer and output (h)

Output layer

softmax

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \text{for } j = 1, \dots, K$$



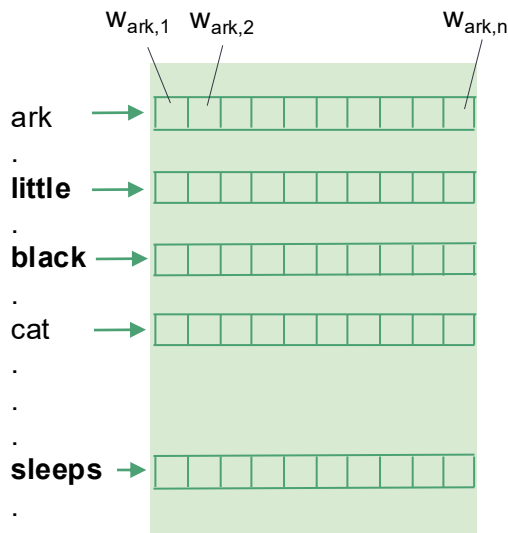
loss (distance)

Input layer
(one hot encoding)

CBOW Details: Word Embeddings

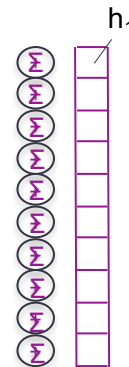
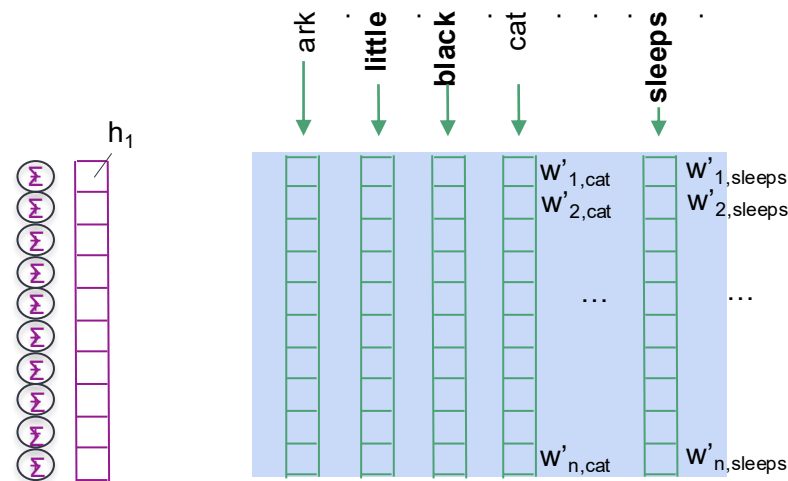
W (VxN)

This hidden layer weight matrix contains context embeddings



W' (NxV)

This output layer weight matrix contains word embeddings



*Word embeddings:
the two matrices (choose one) ...
Representations are the results of many cycles of
forward pass and backpropagation (training)*

Representation learning: learn sub-symbolic representations while performing one or more tasks

Matteo Palmonari

matteo.palmonari@unimib.it

Distributional Semantics & Representation Learning *Word Embeddings & Word2vec*



INSID&S Lab
Interaction and Semantics
for Innovation with Data & Services

Word Embeddings & Word2Vec: Similarity & Regularities

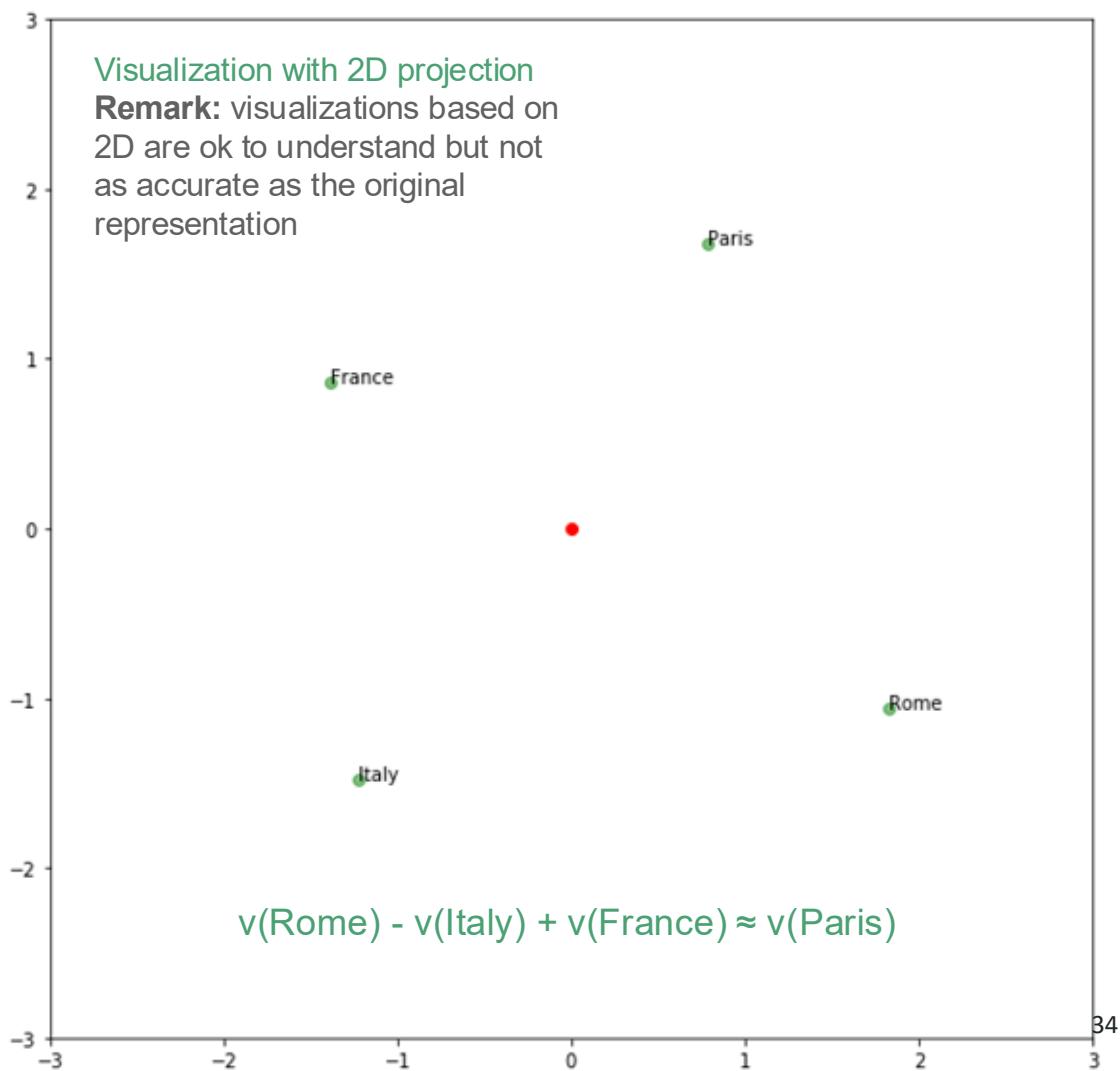
Word Embeddings: Similarity/Distance

Similar words	Distance
brain	0
neuronal	11.25883
cerebral	11.53435
cortical	11.56602
fmri	12.11311
parcellation	12.21351
connectome	12.35784

Words in w2v Vector Space

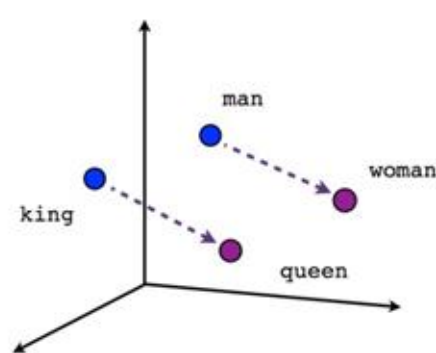
W2v captures linguistic regularities and supports **analogical reasoning**

Italy is to Rome what France is to Paris

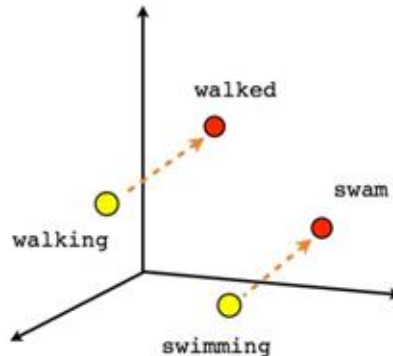


Word2vec: Regularities and Analogies

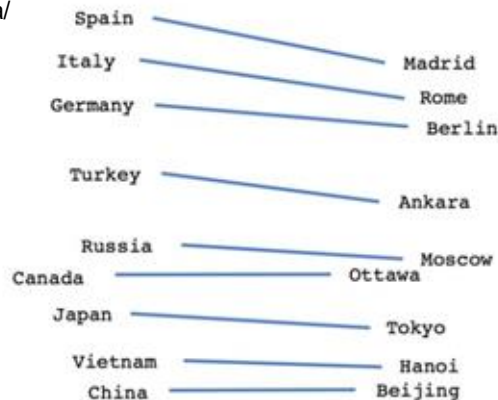
<https://www.distilled.net/resources/a-beginners-guide-to-word2vec-aka-whats-the-opposite-of-canada/>



Male-Female



Verb tense



Country-Capital

Propositional analogies

What is to Spain what Rome is to Italy?

Spain : ?x :: Italy : Rome

Returns a point in the vector space

$NearestNeighbor(v(Rome) - v(Italy) + v(Spain)) = ?x$

$v(Rome) - v(Italy) + v(Spain) \approx v(Madrid)$

Word2vec: Main Features

- The representation of a word in Word2vec is generated by **considering the words that co-occur in its context**
 - predict target word from context or context from target word
- Word2vec represents words in a vector space in which **similar words are close** one to each other
 - Usually via cosine similarity
- Vector components in the vector **do not have** an intuitive interpretation; vector components are difficult to interpret
 - Some works attempted to provide some justifications

Word2vec: a Feature Extraction Method

- Some downstream usage of word vectors
 - Word2vec for feature extraction
 - word $\rightarrow$ word vector
 - set of words $\rightarrow$ combination of word vectors
 - Word2vec for similarity between words
 - Features input to some ML method (e.g., classifier, clustering, etc)
- Word2vec derived methods
 - sentences, documents, etc., e.g., doc2vec
 - graphs, e.g., node2vec, RDF2vec



I paid 100 Euros for a really flavourless food and not so delightful ambience.



Food was fine and I wouldn't say it was the best place I have ever tried.



We loved the food. Menu is perfect in here, something for everyone. Visiting this one again.

Example#1: a simple approach to **sentiment analysis**

<https://medium.com/swlh/sentiment-classification-using-word-embeddings-word2vec-aedf28fbb8ca>

Word Embeddings & Word2Vec: Resources

Word2Vec: Additional Resources

Best easy-to-understand explanation on word2vec

<http://jalanmar.github.io/illustrated-word2vec/>

Word2Vec Tutorial - The Skip-Gram Model

<http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>

Training strategies (from softmax to negative samplings) + tips (e.g., phrase detection: “New York” as one token)

<https://lilianweng.github.io/lil-log/2017/10/15/learning-word-embedding.html>

The backpropagation algorithm for Word2Vec (with nice examples)

<http://www.claudiobellei.com/2018/01/06/backprop-word2vec/>

Python implementation of Word2Vec (tensorflow)

<http://www.claudiobellei.com/2018/01/07/backprop-word2vec-python/>

World Embeddings after Word2Vec

pre-trained embeddings
for word2vec and other
models are available for
download to be used in
downstream NLP tasks

Glove

“Alternative” to word2vec, different optimization procedure. <https://nlp.stanford.edu/projects/glove/>

Probabilistic Word Embeddings

“Each word is a gaussian distribution” <https://arxiv.org/pdf/1412.6623.pdf>

FastText (<https://fasttext.cc/>)

Extends skipgram considering each word has a bag of characters n-grams (subwords); with multilingual embeddings in 157 languages) <https://arxiv.org/pdf/1607.04606.pdf>

key feature: you can get embedding for unseen word based on its subwords since you train on characters n-grams!

Trained on webcrawl and Wikipedia